



INFORME DE TRABAJO EN CLASE

I. PORTADA

Tema:	Implemente el diseño de base de datos
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	Cuarto B
Alumnos participantes:	Luis Miranda
Asignatura:	Base de datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

De los 4 ejercicios enviados resuelva el diseño de base de datos hasta llegar al modelo Físico a saber:

- Modelo Conceptual
- El modelo entidad-relación.
- El modelo relacional (esquema de tablas)
- Modelo físico (oracle)

Cree un informe del desarrollo y suba junto a los scripts

EJERCICIO 1

Enunciado

“Una empresa vende productos a sus clientes. Se necesita conocer los datos personales de los clientes (nombre, apellidos, CI, dirección y fecha de nacimiento). Cada producto tiene un código y un nombre, así como un precio unitario y una marca. Un cliente puede comprar varios productos, y un mismo producto puede ser comprado por varios clientes. Los productos son suministrados por diferentes proveedores. Se debe tener en cuenta que un proveedor puede suministrar diferentes productos y que un producto sólo puede ser suministrado por un proveedor. De cada proveedor se conoce el RUC, nombre y dirección.”

1. Análisis del problema

Del enunciado se identifican primero los objetos principales del negocio:

- **Cliente**
- **Producto**
- **Proveedor**

Además, para representar correctamente la operación de compra, conviene agregar una entidad transaccional:

- **Compra**
- **Detalle_Compra**

Esto se hace porque:

- un cliente puede comprar muchos productos,
- un producto puede ser comprado por muchos clientes,
- esa relación es de tipo **muchos a muchos (N:M)**,
- y en el modelo relacional una relación N:M debe resolverse con una tabla intermedia.

2. Modelo conceptual

2.1 Entidades del negocio

Cliente

Representa a las personas que compran productos.



Atributos:

- CI
- Nombre
- Apellidos
- Dirección
- Fecha de nacimiento

Proveedor

Representa a las empresas o personas que suministran productos.

Atributos:

- RUC
- Nombre
- Dirección

Producto

Representa los bienes que la empresa vende.

Atributos:

- Código
- Nombre
- Precio unitario
- Marca

Compra

Representa una transacción realizada por un cliente.

Atributos:

- IdCompra
- FechaCompra

Detalle_Compra

Representa los productos incluidos en una compra.

Atributos:

- Cantidad
- PrecioVenta

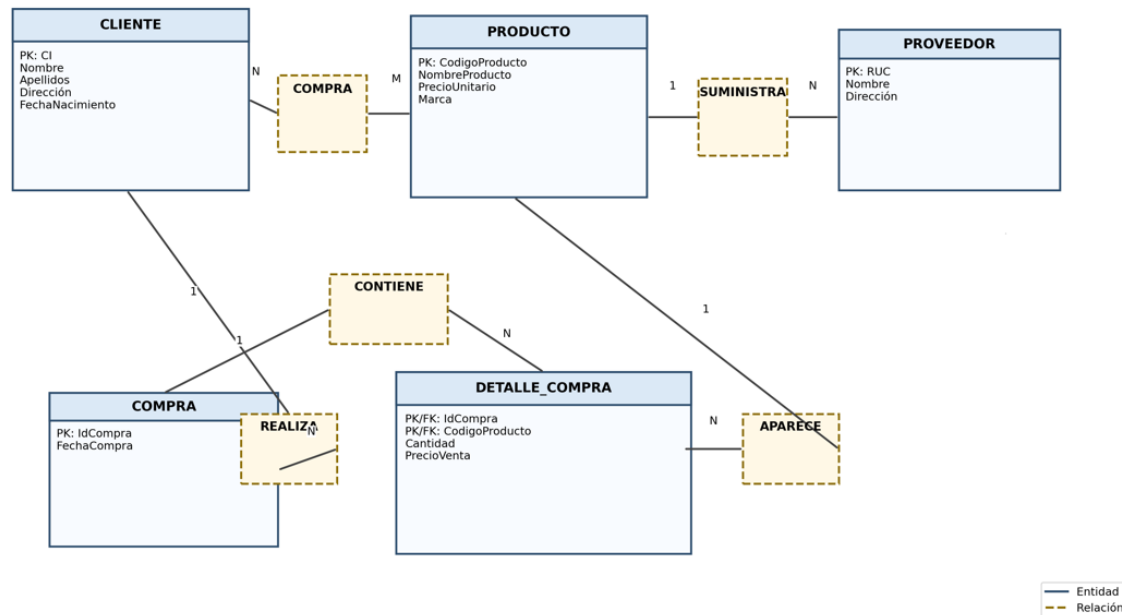
2.2 Reglas de negocio

1. Un **cliente** puede realizar muchas compras.
2. Una **compra** pertenece a un solo cliente.
3. Una **compra** puede incluir uno o varios productos.
4. Un **producto** puede aparecer en muchas compras.
5. Un **proveedor** puede suministrar muchos productos.
6. Un **producto** es suministrado por un solo proveedor.



Diagrama Entidad-Relación (ER) - Modelo conceptual

Caso: empresa que vende productos a clientes y registra proveedores, compras y detalle de compra



3. Modelo Entidad-Relación (ER)

3.1 Entidades con sus claves

CLIENTE

- **CI** (*clave primaria*)
- Nombre
- Apellidos
- Dirección
- FechaNacimiento

PROVEEDOR

- **RUC** (*clave primaria*)
- Nombre
- Dirección

PRODUCTO

- **CódigoProducto** (*clave primaria*)
- NombreProducto
- PrecioUnitario
- Marca

COMPRA

- **IdCompra** (*clave primaria*)
- FechaCompra

DETALLE_COMPRA

- **IdCompra** (*parte de clave primaria y clave foránea*)
- **CódigoProducto** (*parte de clave primaria y clave foránea*)
- Cantidad
- PrecioVenta

3.2 Relaciones y cardinalidades

PROVEEDOR — SUMINISTRA — PRODUCTO

- Un proveedor **suministra** muchos productos



- Un producto es suministrado por un solo proveedor

Cardinalidad:

PROVEEDOR (1) ----- (N) PRODUCTO

CLIENTE — REALIZA — COMPRA

- Un cliente **realiza muchas compras**
- Una compra **corresponde a un solo cliente**

Cardinalidad:

CLIENTE (1) ----- (N) COMPRA

COMPRA — CONTIENE — DETALLE_COMPRA

- Una compra **tiene varios detalles**
- Cada detalle pertenece a una sola compra

Cardinalidad:

COMPRA (1) ----- (N) DETALLE_COMPRA

PRODUCTO — APARECE_EN — DETALLE_COMPRA

- Un producto puede aparecer en muchos detalles
- Cada detalle hace referencia a un solo producto

Cardinalidad:

PRODUCTO (1) ----- (N) DETALLE_COMPRA

Paso del modelo ER al modelo relacional

Regla 1. Cada entidad fuerte se convierte en una tabla

Por lo tanto se crean las tablas:

- CLIENTES
- PROVEEDORES
- PRODUCTOS
- COMPRAS

Regla 2. Relación 1:N

En una relación 1:N, la clave primaria del lado 1 pasa como clave foránea al lado N.

PROVEEDOR 1:N PRODUCTO

La clave primaria de **PROVEEDORES** pasa a **PRODUCTOS**.

Entonces:

- RUC_PROV será FK en PRODUCTOS

CLIENTE 1:N COMPRA

La clave primaria de **CLIENTES** pasa a **COMPRAS**.

Entonces:

- CI_CLI será FK en COMPRAS

Regla 3. Relación N:M

La relación N:M se transforma en una tabla intermedia.

La compra de productos se resuelve con:

- COMPRAS
- DETALLE_COMPRA

En DETALLE_COMPRA se almacenan:

- ID_COMPRA
- COD_PROD
- CANTIDAD
- PRECIO_VENTA

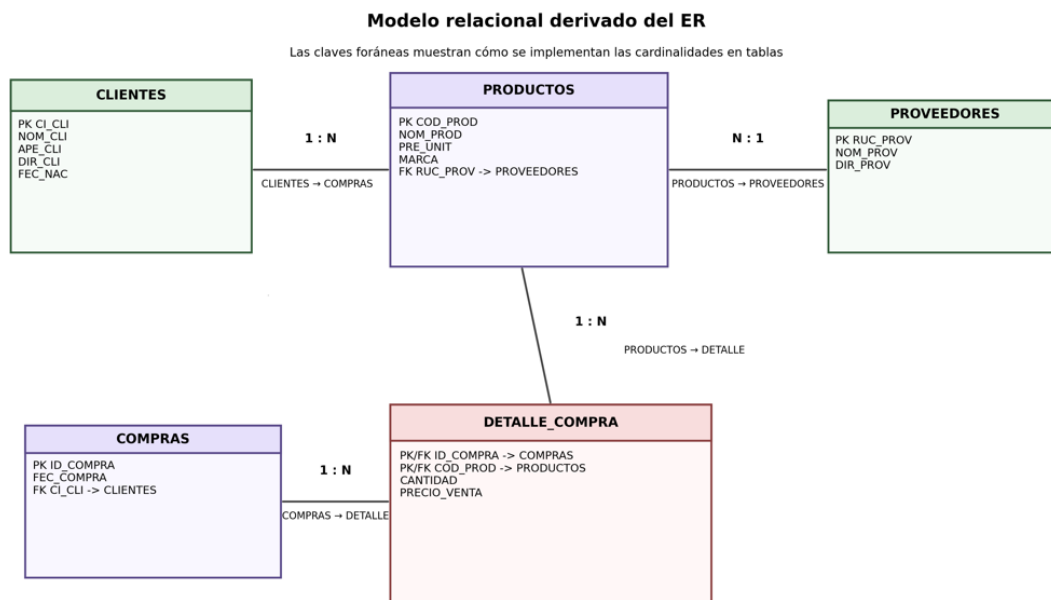
La clave primaria será compuesta:

- (ID_COMPRA, COD_PROD)



5. Modelo relacional

Esquema relacional final



6. Modelo físico en Oracle

Aquí ya se definen:

- tipos de datos Oracle,
- restricciones,
- nombres de claves primarias,
- claves foráneas,
- validaciones con CHECK.

7.1 Eliminación previa de tablas si ya existen

Esto sirve para volver a ejecutar el script.

-- Ejecutar solo si las tablas ya existen

```
DROP TABLE DETALLE_COMPRA CASCADE CONSTRAINTS;
```

```
DROP TABLE COMPRAS CASCADE CONSTRAINTS;
```

```
DROP TABLE PRODUCTOS CASCADE CONSTRAINTS;
```

```
DROP TABLE CLIENTES CASCADE CONSTRAINTS;
```

```
DROP TABLE PROVEEDORES CASCADE CONSTRAINTS;
```

7.2 Creación de tablas

.....

DDL

8. DML en Oracle

8.1 Inserción de datos en PROVEEDORES

8.2 Inserción de datos en CLIENTES

8.3 Inserción de datos en PRODUCTOS

8.4 Inserción de datos en COMPRAS

8.5 Inserción de datos en DETALLE_COMPRA

8.6 Confirmar cambios



9. Consultas de comprobación

Ver clientes

Consulta integrada

Muestra qué compró cada cliente.

SELECT

c.CI_CLI,
c.NOM_CLI,
c.APE_CLI,
co.ID_COMPRA,
co.FEC_COMPRA,
p.COD_PROD,
p.NOM_PROD,
d.CANTIDAD,
d.PRECIO_VENTA

FROM CLIENTES c

JOIN COMPRAS co ON c.CI_CLI = co.CI_CLI

JOIN DETALLE_COMPRA d ON co.ID_COMPRA = d.ID_COMPRA

JOIN PRODUCTOS p ON d.COD_PROD = p.COD_PROD

ORDER BY co.ID_COMPRA;

10. Operaciones DML adicionales

10.1 UPDATE

Cambiar dirección de un cliente

Cambiar precio unitario de un producto

Cambiar marca de un producto

10.2 DELETE

Eliminar un detalle de compra

Eliminar una compra si ya no tiene detalles

10.3 Guardar cambios

11. Pruebas de errores de integridad en Oracle

Error 1: clave primaria duplicada

Error 2: clave foránea inexistente

Error 3: eliminación de registro padre con hijos

Error 4: violación de CHECK

11. SCRIPT

```
SQL> CREATE USER CLINICA IDENTIFIED BY CLINICA123;  
CREATE USER CLINICA IDENTIFIED BY CLINICA123
```

```
SQL> CREATE USER EMPRESA IDENTIFIED BY EMPRESA123;
```

Usuario creado.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO- JULIO 2026



SQL> GRANT CONNECT, RESOURCE, UNLIMITED TABLESPACE TO
EMPRESA;

Concesión terminada correctamente.

SQL> CONNECT EMPRESA;

Introduzca la contraseña:

Conectado.

```
SQL> CREATE TABLE PROVEEDOR
  2  (RUC_PRO VARCHAR2(13) PRIMARY KEY,
  3  NOM_PRO VARCHAR2(100) NOT NULL,
  4  DIR_PRO VARCHAR2(100));
```

Tabla creada.

```
SQL> CREATE TABLE PRODUCTO
  2  (COD_PROD NUMBER PRIMARY KEY,
  3  NOM_PROD VARCHAR2(100) NOT NULL,
  4  PRE_UNI_PROD NUMBER(10,2) CHECK (PRE_UNI_PROD > 0),
  5  MAR_PROD VARCHAR2(50),
  6  RUC_PRO VARCHAR2(13),
  7  CONSTRAINT FK_PRODUCTO_PROVEEDOR FOREIGN KEY (RUC_PRO)
REFERENCES PROVEEDOR(RUC_PRO));
```

Tabla creada.

```
SQL> CREATE TABLE CLIENTE
  2  (CI_CLI VARCHAR2(10) PRIMARY KEY,
  3  NOM_CLI VARCHAR2(50) NOT NULL,
  4  APE_CLI VARCHAR2(50) NOT NULL,
  5  DIR_CLI VARCHAR2(100),
  6  FEC_NAC_CLI DATE);
```

Tabla creada.

```
SQL> CREATE TABLE COMPRA
  2  (ID_COM NUMBER PRIMARY KEY,
  3  CI_CLI VARCHAR2(10),
  4  COD_PROD NUMBER,
  5  CONSTRAINT FK_COMPRA_CLIENTE FOREIGN KEY (CI_CLI)
REFERENCES CLIENTE(CI_CLI),
  6  CONSTRAINT FK_COMPRA_PRODUCTO FOREIGN KEY (COD_PROD)
REFERENCES PRODUCTO(COD_PROD));
```

Tabla creada.

SQL> COMMIT;

Confirmación terminada.

```
SQL> INSERT INTO PROVEEDOR
  2  VALUES ('1790012345001', 'Tecno', 'Quito');
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO- JULIO 2026



1 fila creada.

```
SQL> INSERT INTO
```

```
2 PROVEEDOR VALUES ('1790098766001', 'Alumco',  
'Guayaquil');
```

1 fila creada.

```
SQL> INSERT INTO
```

```
2 PROVEEDOR VALUES ('1790045678001', 'Tech', 'Ambato');
```

1 fila creada.

```
SQL> COMMIT;
```

Confirmación terminada.

```
SQL> INSERT INTO
```

```
2  
SQL> INSERT INTO PRODUCTO
```

```
2 VALUES (101, 'Laptop', 900, 'DELL', '1790012345001');
```

1 fila creada.

```
SQL> INSERT INTO PRODUCTO
```

```
2 VALUES (102, 'Tablet', 300, 'Samsung',  
'1790045678001');
```

1 fila creada.

```
SQL> INSERT INTO
```

```
2 PRODUCTO VALUES (103, 'Proyector', 450, 'Epson',  
'1790098766001');
```

1 fila creada.

```
SQL> COMMIT;
```

Confirmación terminada.

```
SQL> INSERT INTO CLIENTE
```

```
2 VALUES ('180550', 'Juan', 'Lopez', 'Ambato',  
TO_DATE('2000-05-12', 'YYYY-MM-DD'));
```

1 fila creada.

```
SQL> INSERT INTO CLIENTE
```

```
2 VALUES ('180560', 'Carlos', 'Perez', 'Cuenca',  
TO_DATE('1999-11-03', 'YYYY-MM-DD'));
```

1 fila creada.

```
SQL> INSERT INTO CLIENTE
```



```
2 VALUES ('180570', 'Andrea', 'Morales', 'Quito',  
TO_DATE('2001-02-20', 'YYYY-MM-DD'));
```

1 fila creada.

```
SQL> INSERT INTO COMPRA  
2 VALUES (1, '180570', 101);
```

1 fila creada.

```
SQL> INSERT INTO COMPRA  
2 VALUES (2, '180560', 102);
```

1 fila creada.

```
SQL> INSERT INTO COMPRA  
2 VALUES (3, '180550', 101);
```

1 fila creada.

```
SQL> COMMIT;
```

EJERCICIO 2

Enunciado

Se desea diseñar la base de datos de un Instituto. En la base de datos se desea guardar los datos de los profesores del Instituto (ci, nombre, apellido, dirección y teléfono). Cada alumno está matriculado en uno o varios módulos que se imparten en distinto horario, y en diferente aula, y en el mismo modulo pueden estar varios alumnos matriculados. De cada alumno se desea guardar el id, nombre, apellido y fecha de nacimiento. Los profesores pueden impartir varios módulos, pero un módulo sólo puede ser impartido por un profesor. Cada módulo tiene un código y un nombre. Los profesores tienen un jefe de supervisión que también es profesor, pero que además es el encargado de supervisar al resto de profesores.

1. Análisis del problema

Se identifican los objetos principales:

- **Profesor**
- **Alumno**
- **Modulo**

Para resolver la relación muchos a muchos:

- **Matricula (tabla intermedia)**

Porque:



- Un alumno puede inscribirse en varios módulos
- Un módulo puede tener varios alumnos
- Es una relación **N:M**, que necesita tabla intermedia

2. Modelo conceptual

2.1 Entidades del negocio

Profesor

Atributos:

- CI
- Nombre
- Apellido
- Dirección
- Teléfono
- CI_Jefe (auto-relación)

Alumno

Atributos:

- Id_Alumno
- Nombre
- Apellido
- Fecha_Nacimiento

Modulo

Atributos:

- Cod_Mod
- Nombre_Mod
- CI_Profesor

Matricula

Atributos:

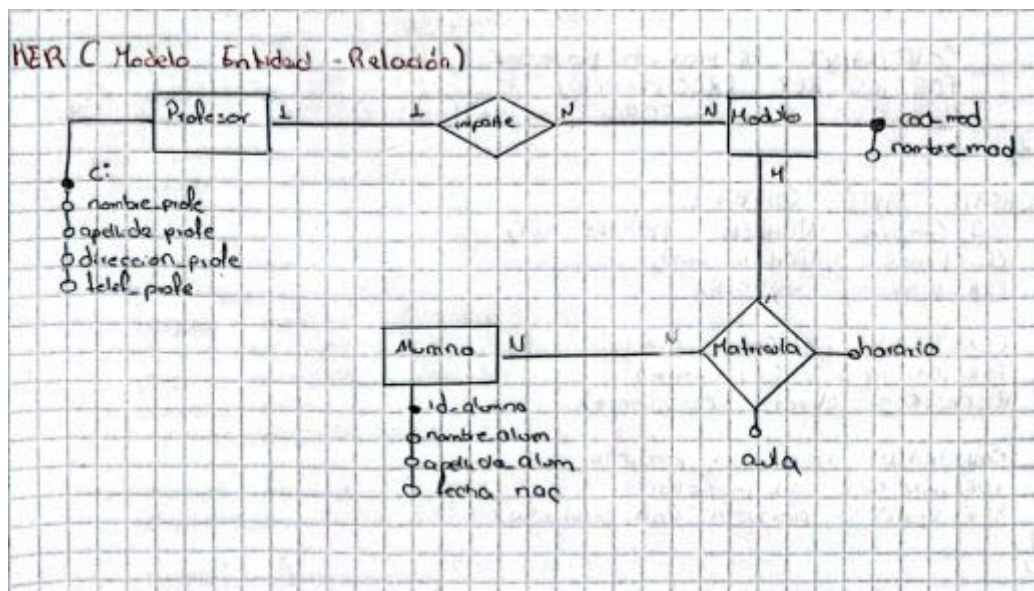
- Id_Alumno
- Cod_Mod
- Horario
- Aula

2.2 Reglas de negocio

1. Un profesor puede dictar varios módulos
2. Un módulo es dictado por un solo profesor
3. Un alumno puede matricularse en varios módulos



4. Un módulo puede tener varios alumnos
5. Un profesor puede tener un jefe (auto-relación)
6. Un profesor puede supervisar a otros profesores



3. Modelo Entidad-Relación (ER)

3.1 Entidades con claves

PROFESOR

- CI (PK)
- Nombre
- Apellido
- Dirección
- Teléfono
- CI_Jefe (FK)

ALUMNO

- Id_Alumno (PK)
- Nombre
- Apellido
- Fecha_Nac

MODULO

- Cod_Mod (PK)
- Nombre_Mod
- CI_Profesor (FK)

MATRICULA



- Id_Alumno (PK, FK)
- Cod_Mod (PK, FK)
- Horario
- Aula

3.2 Relaciones y cardinalidades

PROFESOR — DICTA — MODULO

- Un profesor dicta muchos módulos
- Un módulo tiene un solo profesor

Cardinalidad:

PROFESOR (1) ----- (N) MODULO

ALUMNO — SE MATRICULA — MODULO

- Un alumno puede estar en varios módulos
- Un módulo tiene varios alumnos

Cardinalidad:

ALUMNO (N) ----- (M) MODULO

(resuelto con MATRICULA)

PROFESOR — SUPERVISA — PROFESOR (Auto-relación)

- Un profesor puede tener un jefe
- Un jefe puede supervisar varios profesores

Cardinalidad:

PROFESOR (1) ----- (N) PROFESOR

Paso del modelo ER al modelo relacional

Regla 1: Entidades → Tablas

Se crean:

- PROFESOR
- ALUMNO
- MODULO
- MATRICULA

Regla 2: Relación 1:N

PROFESOR → MODULO

- CI pasa como FK a MODULO



CI_PROFESOR

PROFESOR → PROFESOR (auto-relación)

CI_JEFE es FK en PROFESOR

Regla 3: Relación N:M

Se crea tabla intermedia:

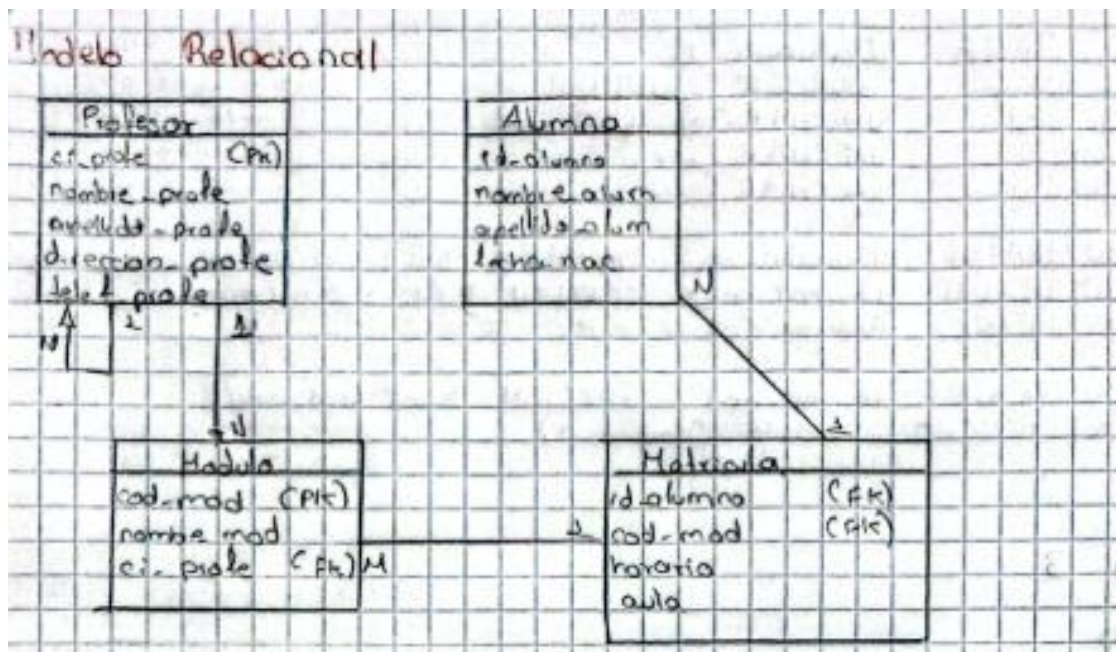
MATRICULA

Con:

- ID_ALUMNO
- COD_MOD
- HORARIO
- AULA

Clave primaria compuesta:
(ID_ALUMNO, COD_MOD)

5. Modelo relacional



6. Modelo físico en Oracle (DDL)

```
CREATE TABLE PROFESOR (  
  ci VARCHAR2(10) PRIMARY KEY,  
  nombre VARCHAR2(50) NOT NULL,  
  apellido VARCHAR2(50) NOT NULL,  
  direccion VARCHAR2(100),
```



```
telefono VARCHAR2(15),
ci_jefe VARCHAR2(10),
CONSTRAINT fk_prof_jefe FOREIGN KEY (ci_jefe)
REFERENCES PROFESOR(ci)
);

CREATE TABLE ALUMNO (
    id_alumno NUMBER PRIMARY KEY,
    nombre VARCHAR2(50) NOT NULL,
    apellido VARCHAR2(50) NOT NULL,
    fecha_nac DATE
);

CREATE TABLE MODULO (
    cod_mod VARCHAR2(10) PRIMARY KEY,
    nombre_mod VARCHAR2(100) NOT NULL,
    ci_profesor VARCHAR2(10) NOT NULL,
    CONSTRAINT fk_mod_prof FOREIGN KEY (ci_profesor)
REFERENCES PROFESOR(ci)
);

CREATE TABLE MATRICULA (
    id_alumno NUMBER NOT NULL,
    cod_mod VARCHAR2(10) NOT NULL,
    horario VARCHAR2(50),
    aula VARCHAR2(20),
    CONSTRAINT pk_matricula PRIMARY KEY (id_alumno, cod_mod),
    CONSTRAINT fk_mat_alum FOREIGN KEY (id_alumno)
REFERENCES ALUMNO(id_alumno),
    CONSTRAINT fk_mat_mod FOREIGN KEY (cod_mod)
REFERENCES MODULO(cod_mod)
);
```

8. DML (Inserciones)

8.1 PROFESOR

```
INSERT INTO PROFESOR VALUES
('0101','Juan','Perez','Ambato','0999999999',NULL);
INSERT INTO PROFESOR VALUES
('0102','Maria','Lopez','Quito','0988888888','0101');
```

8.2 ALUMNO

```
INSERT INTO ALUMNO VALUES (1,'Carlos','Mendez',DATE '2000-
05-10');
INSERT INTO ALUMNO VALUES (2,'Ana','Torres',DATE '2001-08-
15');
```

8.3 MODULO



```
INSERT INTO MODULO VALUES ('M01','Base de Datos','0101');  
INSERT INTO MODULO VALUES ('M02','Programacion','0102');
```

8.4 MATRICULA

```
INSERT INTO MATRICULA VALUES (1,'M01','08:00-10:00','A1');  
INSERT INTO MATRICULA VALUES (2,'M02','10:00-12:00','B1');
```

8.5 Confirmar

```
COMMIT;
```

9. Consulta integrada

```
SELECT  
    a.id_alumno,  
    a.nombre,  
    a.apellido,  
    m.cod_mod,  
    m.nombre_mod,  
    ma.horario,  
    ma.aula  
FROM ALUMNO a  
JOIN MATRICULA ma ON a.id_alumno = ma.id_alumno  
JOIN MODULO m ON ma.cod_mod = m.cod_mod  
ORDER BY a.id_alumno;
```

10. Operaciones DML

UPDATE

```
UPDATE ALUMNO SET nombre='Luis' WHERE id_alumno=1;  
UPDATE MODULO SET nombre_mod='BD Avanzada' WHERE  
cod_mod='M01';
```

DELETE

```
DELETE FROM MATRICULA WHERE id_alumno=1 AND cod_mod='M01';
```

11. SCRIPT

```
SQL> CREATE USER INSTITUTO IDENTIFIED BY INSTITUTO123;
```

Usuario creado.

```
SQL> GRANT CONNECT, RESOURCE, UNLIMITED TABLESPACE TO  
INSTITUTO;
```

Concesión terminada correctamente.

```
SQL> CONNECT INSTITUTO;  
Introduzca la contraseña:
```



Conectado.

```
SQL> CREATE TABLE PROFESOR
  2 (CI_PROF VARCHAR2(10) PRIMARY KEY,
  3 NOM_PROF VARCHAR2(50) NOT NULL,
  4 APE_PROF VARCHAR2(50) NOT NULL,
  5 DIR_PROF VARCHAR2(100),
  6 TEL_PROF VARCHAR2(15),
  7 CI_JEFE VARCHAR2(10),
  8 CONSTRAINT FK_PROF_JEFE FOREIGN KEY (CI_JEFE) REFERENCES
PROFESOR(CI_PROF));
```

Tabla creada.

```
SQL> CREATE TABLE ALUMNO
  2 (ID_ALU NUMBER PRIMARY KEY,
  3 NOM_ALU VARCHAR2(50) NOT NULL,
  4 APE_ALU VARCHAR2(50) NOT NULL,
  5 FEC_NAC_ALU DATE);
```

Tabla creada.

```
SQL> CREATE TABLE MODULO
  2 (COD_MOD VARCHAR2(10) PRIMARY KEY,
  3 NOM_MOD VARCHAR2(50) NOT NULL,
  4 HORARIO_MOD VARCHAR2(50),
  5 AULA_MOD VARCHAR2(20),
  6 CI_PROF VARCHAR2(10),
  7 CONSTRAINT FK_MOD_PROF FOREIGN KEY (CI_PROF) REFERENCES
PROFESOR(CI_PROF));
```

Tabla creada.

```
SQL> CREATE TABLE MATRICULA
  2 (ID_ALU NUMBER,
  3 COD_MOD VARCHAR2(10),
  4 PRIMARY KEY (ID_ALU, COD_MOD),
  5 CONSTRAINT FK_MAT_ALU FOREIGN KEY (ID_ALU) REFERENCES
ALUMNO(ID_ALU),
  6 CONSTRAINT FK_MAT_MOD FOREIGN KEY (COD_MOD) REFERENCES
MODULO(COD_MOD));
```

Tabla creada.

```
SQL> COMMIT;
```

Confirmación terminada.

```
SQL> INSERT INTO PROFESOR
  2 VALUES ('180111', 'Ana', 'Torres', 'Ficoa', '098111', NULL);
```

1 fila creada.



SQL> INSERT INTO PROFESOR

2 VALUES ('180222', 'Luis', 'Castro', 'Centro', '098222', '180111');

1 fila creada.

SQL> INSERT INTO PROFESOR

2 VALUES ('180333', 'Maria', 'Gomez', 'Huachi', '098333', '180111');

1 fila creada.

SQL> INSERT INTO ALUMNO

2 VALUES (1, 'Pedro', 'Ruiz', TO_DATE('2003-05-14', 'YYYY-MM-DD'));

1 fila creada.

SQL> INSERT INTO ALUMNO

2 VALUES (2, 'Sofia', 'Luna', TO_DATE('2004-08-21', 'YYYY-MM-DD'));

1 fila creada.

SQL> INSERT INTO ALUMNO

2 VALUES (3, 'Diego', 'Vega', TO_DATE('2002-11-10', 'YYYY-MM-DD'));

1 fila creada.

SQL> INSERT INTO MODULO

2 VALUES ('M101', 'Bases de Datos', 'Lunes 08:00', 'Lab 1', '180222');

1 fila creada.

SQL> INSERT INTO MODULO

2 VALUES ('M102', 'Programacion', 'Martes 10:00', 'Lab 2', '180333');

1 fila creada.

SQL> INSERT INTO MODULO

2 VALUES ('M103', 'Redes', 'Jueves 14:00', 'Lab 3', '180222');

1 fila creada.

SQL> INSERT INTO MATRICULA

2 VALUES (1, 'M101');

1 fila creada.

SQL> INSERT INTO MATRICULA

2 VALUES (1, 'M102');

1 fila creada.



```
SQL> INSERT INTO MATRICULA  
2 VALUES (2, 'M101');
```

1 fila creada.

```
SQL> COMMIT;
```

Confirmación terminada.

EJERCICIO 3

Enunciado

Se desea informatizar la gestión de una tienda informática. La tienda dispone de una serie de productos que se pueden vender a los clientes.

"De cada producto informático se desea guardar el código, descripción, precio y número de existencias. De cada cliente se desea guardar el código, nombre, apellido, dirección y número de teléfono.

Un cliente puede comprar varios productos en la tienda y un mismo producto puede ser comprado por varios clientes, con diferente número de la compra, la fecha en la que se ha comprado el artículo y la forma de pago.

La tienda tiene contactos con varios proveedores que son los que entregan los productos. Un mismo producto puede ser entregado por varios proveedores y un proveedor entrega varios productos, se debe registrar el id de la entrega, la fecha de la entrega, la cantidad suministrada de ese producto, y el precio al que se adquiere el producto. De cada proveedor se desea guardar el código, nombre, apellidos, dirección, provincia y número de teléfono.

1. Análisis del problema

Se identifican las entidades principales:

- **Cliente**
- **Producto**
- **Proveedor**

Y las entidades intermedias:

- **Compra**
- **Entrega**

Porque:

- Cliente ↔ Producto → relación **N:M**
- Proveedor ↔ Producto → relación **N:M**
- Estas relaciones se resuelven con tablas intermedias



2. Modelo conceptual

2.1 Entidades del negocio

Cliente

Atributos:

- Cod_Cli
- Nombre
- Apellido
- Dirección
- Teléfono

Producto

Atributos:

- Cod_Prod
- Descripción
- Precio
- Existencias

Proveedor

Atributos:

- Cod_Prov
- Nombre
- Apellido
- Dirección
- Provincia
- Teléfono

Compra

Atributos:

- Num_Compra
- Cod_Cli
- Cod_Prod
- Fecha_Compra
- Forma_Pago

Entrega

Atributos:

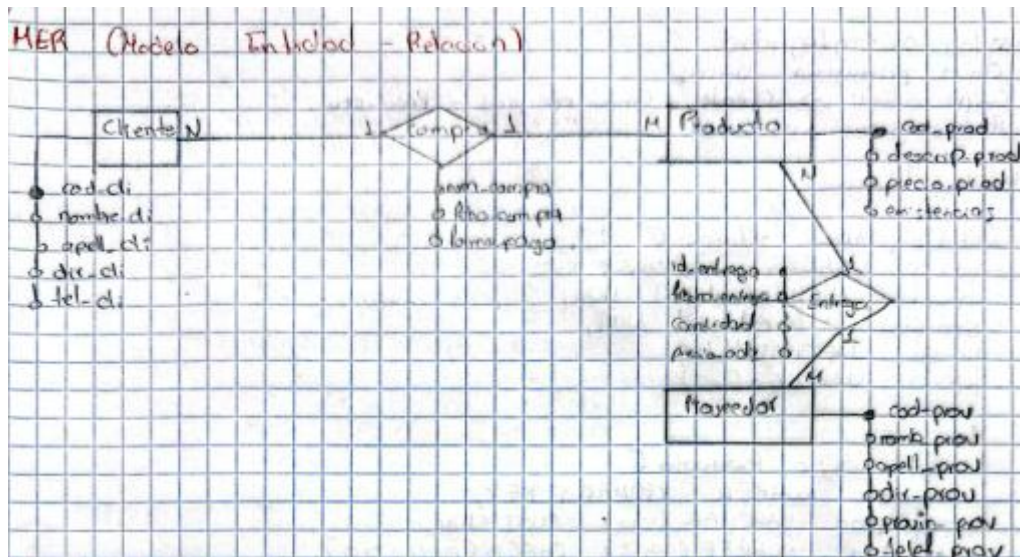
- Id_Entrega
- Cod_Prov



- Cod_Prod
- Fecha_Entrega
- Cantidad
- Precio_Adq

2.2 Reglas de negocio

1. Un cliente puede realizar varias compras
2. Un producto puede ser comprado por varios clientes
3. Un proveedor puede suministrar varios productos
4. Un producto puede ser suministrado por varios proveedores
5. Toda compra debe estar asociada a un cliente y un producto
6. Toda entrega debe estar asociada a un proveedor y un producto



3. Modelo Entidad-Relación (ER)

3.1 Entidades con claves

CLIENTE

- Cod_Cli (PK)
- Nombre
- Apellido
- Dirección
- Teléfono

PRODUCTO

- Cod_Prod (PK)
- Descripción
- Precio
- Existencias



PROVEEDOR

- Cod_Prov (PK)
- Nombre
- Apellido
- Dirección
- Provincia
- Teléfono

COMPRA

- Num_Compra (PK)
- Cod_Cli (FK)
- Cod_Prod (FK)
- Fecha_Compra
- Forma_Pago

ENTREGA

- Id_Entrega (PK)
- Cod_Prov (FK)
- Cod_Prod (FK)
- Fecha_Entrega
- Cantidad
- Precio_Adq

3.2 Relaciones y cardinalidades

CLIENTE — COMPRA — PRODUCTO

- Un cliente compra varios productos
- Un producto es comprado por varios clientes

Cardinalidad:

CLIENTE (N) ----- (M) PRODUCTO

(resuelto con COMPRA)

PROVEEDOR — ENTREGA — PRODUCTO

- Un proveedor entrega varios productos
- Un producto puede ser entregado por varios proveedores

Cardinalidad:

PROVEEDOR (N) ----- (M) PRODUCTO

(resuelto con ENTREGA)



Paso del modelo ER al modelo relacional

Regla 1: Entidades → Tablas

Se crean:

- CLIENTE
- PRODUCTO
- PROVEEDOR
- COMPRA
- ENTREGA

Regla 2: Relación N:M

CLIENTE ↔ PRODUCTO

Se crea tabla intermedia:

COMPRA

Contiene:

- NUM_COMPRA
- COD_CLI
- COD_PROD
- FECHA
- FORMA_PAGO

PROVEEDOR ↔ PRODUCTO

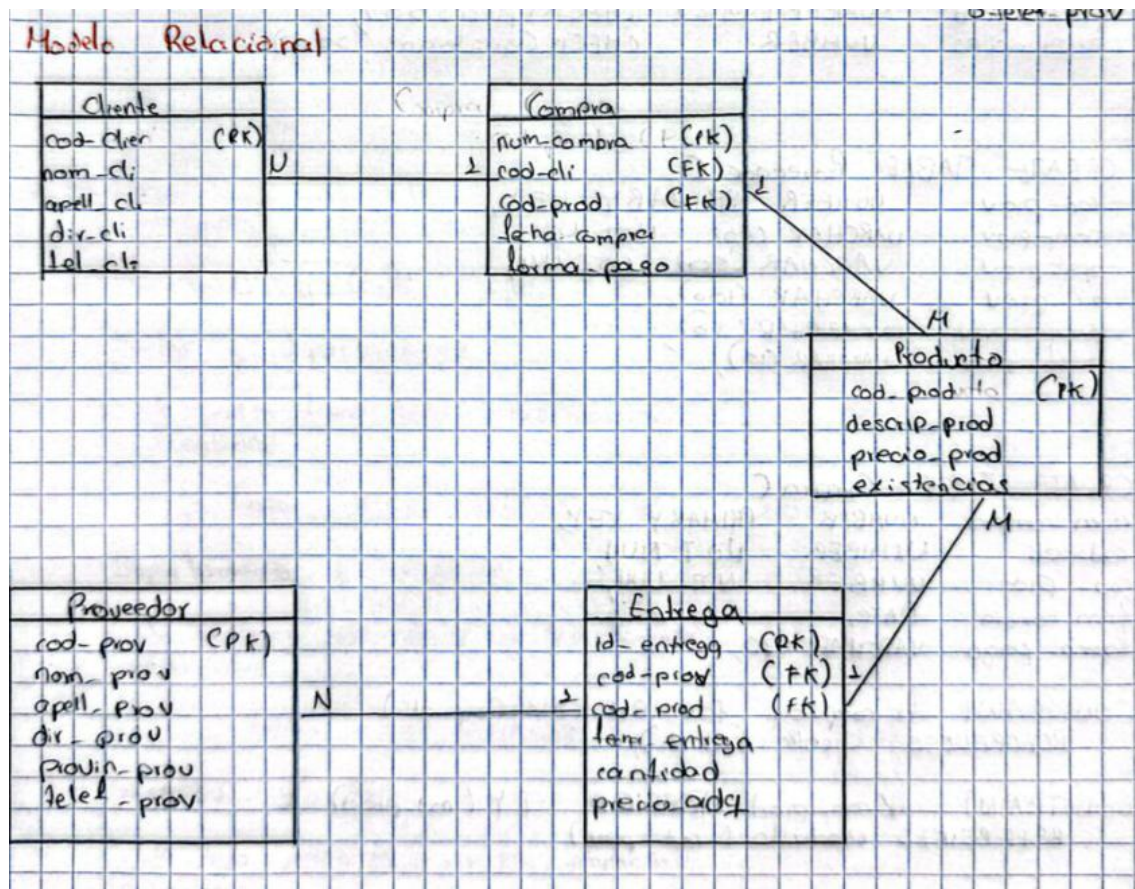
Se crea:

ENTREGA

Contiene:

- ID_ENTREGA
- COD_PROV
- COD_PROD
- FECHA
- CANTIDAD
- PRECIO_ADQ

5. Modelo relacional



6. Modelo físico en Oracle (DDL)

```
CREATE TABLE CLIENTE (  
    cod_cli    NUMBER PRIMARY KEY,  
    nombre     VARCHAR2(50) NOT NULL,  
    apellido   VARCHAR2(50) NOT NULL,  
    direccion  VARCHAR2(100),  
    telefono   VARCHAR2(15)  
);  
  
CREATE TABLE PRODUCTO (  
    cod_prod   NUMBER PRIMARY KEY,  
    descripcion VARCHAR2(200) NOT NULL,  
    precio     NUMBER(10,2) CHECK (precio > 0),  
    existencias NUMBER CHECK (existencias >= 0)  
);  
  
CREATE TABLE PROVEEDOR (  
    cod_prov   NUMBER PRIMARY KEY,  
    nombre     VARCHAR2(50) NOT NULL,  
    apellido   VARCHAR2(50) NOT NULL,  
    direccion  VARCHAR2(100),  
    provincia  VARCHAR2(50),  
    telefono   VARCHAR2(15)  
);
```



```
CREATE TABLE COMPRA (  
    num_compra    NUMBER PRIMARY KEY,  
    cod_cli       NUMBER NOT NULL,  
    cod_prod      NUMBER NOT NULL,  
    fecha_compra  DATE,  
    forma_pago    VARCHAR2(30),  
    CONSTRAINT fk_comp_cli FOREIGN KEY (cod_cli)  
        REFERENCES CLIENTE(cod_cli),  
    CONSTRAINT fk_comp_prod FOREIGN KEY (cod_prod)  
        REFERENCES PRODUCTO(cod_prod)  
);
```

```
CREATE TABLE ENTREGA (  
    id_entrega    NUMBER PRIMARY KEY,  
    cod_prov      NUMBER NOT NULL,  
    cod_prod      NUMBER NOT NULL,  
    fecha_entrega DATE,  
    cantidad      NUMBER CHECK (cantidad > 0),  
    precio_adq    NUMBER(10,2) CHECK (precio_adq > 0),  
    CONSTRAINT fk_ent_prov FOREIGN KEY (cod_prov)  
        REFERENCES PROVEEDOR(cod_prov),  
    CONSTRAINT fk_ent_prod FOREIGN KEY (cod_prod)  
        REFERENCES PRODUCTO(cod_prod)  
);
```

8. DML (Inserciones)

CLIENTE

```
INSERT INTO CLIENTE VALUES  
(1, 'Juan', 'Perez', 'Ambato', '099999999');  
INSERT INTO CLIENTE VALUES  
(2, 'Ana', 'Lopez', 'Quito', '098888888');
```

PRODUCTO

```
INSERT INTO PRODUCTO VALUES (1, 'Laptop', 800, 10);  
INSERT INTO PRODUCTO VALUES (2, 'Mouse', 20, 50);
```

PROVEEDOR

```
INSERT INTO PROVEEDOR VALUES  
(1, 'Carlos', 'Ruiz', 'Guayaquil', 'Guayas', '097777777');
```

COMPRA

```
INSERT INTO COMPRA VALUES (1, 1, 1, SYSDATE, 'Efectivo');  
INSERT INTO COMPRA VALUES (2, 2, 2, SYSDATE, 'Tarjeta');
```

ENTREGA



```
INSERT INTO ENTREGA VALUES (1,1,1,SYSDATE,5,700);
```

Confirmar

```
COMMIT;
```

9. Consulta integrada

```
SELECT
    c.nombre,
    c.apellido,
    p.descripcion,
    co.fecha_compra,
    co.forma_pago
FROM CLIENTE c
JOIN COMPRA co ON c.cod_cli = co.cod_cli
JOIN PRODUCTO p ON co.cod_prod = p.cod_prod;
```

10. Operaciones DML

UPDATE

```
UPDATE PRODUCTO SET precio = 850 WHERE cod_prod = 1;
UPDATE CLIENTE SET direccion = 'Latacunga' WHERE cod_cli = 1;
```

DELETE

```
DELETE FROM COMPRA WHERE num_compra = 1;
```

11.SCRYPT

```
SQL> CONNECT TIENDA_INFORMATICA;
```

Introduzca la contraseña:

Conectado.

```
SQL> CREATE TABLE CLIENTE_TIENDA
2 (COD_CLI VARCHAR2(10) PRIMARY KEY,
3  NOM_CLI VARCHAR2(50) NOT NULL,
4  APE_CLI VARCHAR2(50) NOT NULL,
5  DIR_CLI VARCHAR2(100),
6  TEL_CLI VARCHAR2(15));
```

Tabla creada.

```
SQL> CREATE TABLE PROD_TIENDA
2 (COD_PROD VARCHAR2(10) PRIMARY KEY,
3  DESC_PROD VARCHAR2(100) NOT NULL,
4  PRE_PROD NUMBER(10,2) CHECK (PRE_PROD > 0),
5  STOCK_PROD NUMBER CHECK (STOCK_PROD >= 0));
```

Tabla creada.



```
SQL> CREATE TABLE PROV_TIENDA
2 (COD_PROV VARCHAR2(10) PRIMARY KEY,
3 NOM_PROV VARCHAR2(50) NOT NULL,
4 APE_PROV VARCHAR2(50) NOT NULL,
5 DIR_PROV VARCHAR2(100),
6 PROVINCIA_PROV VARCHAR2(50),
7 TEL_PROV VARCHAR2(15));
```

Tabla creada.

```
SQL> CREATE TABLE COMPRA_TIENDA
2 (NUM_COMPRA NUMBER,
3 COD_PROD VARCHAR2(10),
4 COD_CLI VARCHAR2(10),
5 FEC_COM DATE,
6 FOR_PAGO VARCHAR2(20),
7 PRIMARY KEY (NUM_COMPRA, COD_PROD),
8 CONSTRAINT FK_CT_CLI FOREIGN KEY (COD_CLI) REFERENCES
CLIENTE_TIENDA(COD_CLI),
9 CONSTRAINT FK_CT_PROD FOREIGN KEY (COD_PROD) REFERENCES
PROD_TIENDA(COD_PROD));
```

Tabla creada.

```
SQL> CREATE TABLE ENTREGA_TIENDA
2 (ID_ENTREGA NUMBER,
3 COD_PROD VARCHAR2(10),
4 COD_PROV VARCHAR2(10),
5 FEC_ENT DATE,
6 CANTIDAD_ENT NUMBER,
7 PRE_ADQ NUMBER(10,2) CHECK (PRE_ADQ > 0),
8 PRIMARY KEY (ID_ENTREGA, COD_PROD),
9 CONSTRAINT FK_ET_PROV FOREIGN KEY (COD_PROV) REFERENCES
PROV_TIENDA(COD_PROV),
10 CONSTRAINT FK_ET_PROD FOREIGN KEY (COD_PROD) REFERENCES
PROD_TIENDA(COD_PROD));
```

Tabla creada.

```
SQL> COMMIT;
```

Confirmación terminada.

```
SQL> INSERT INTO CLIENTE_TIENDA
2 VALUES ('C001', 'Martin', 'Vargas', 'Ficoa', '0991234567');
```

1 fila creada.

```
SQL> INSERT INTO CLIENTE_TIENDA
2 VALUES ('C002', 'Valeria', 'Silva', 'Huachi Chico', '0987654321');
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO– JULIO 2026



1 fila creada.

```
SQL> INSERT INTO CLIENTE_TIENDA  
2 VALUES ('C003', 'Andres', 'Morales', 'Ingahurco', '0971122334');
```

1 fila creada.

```
SQL> INSERT INTO PROD_TIENDA  
2 VALUES ('P10', 'Monitor 24', 150.00, 25);
```

1 fila creada.

```
SQL> INSERT INTO PROD_TIENDA  
2 VALUES ('P11', 'Teclado Mecanico', 45.50, 40);
```

1 fila creada.

```
SQL> INSERT INTO PROD_TIENDA  
2 VALUES ('P12', 'Mouse Inalambrico', 20.00, 60);
```

1 fila creada.

```
SQL> INSERT INTO PROV_TIENDA  
2 VALUES ('PR01', 'Carlos', 'Mendez', 'La Magdalena', 'Pichincha', '022111222');
```

1 fila creada.

```
SQL> INSERT INTO PROV_TIENDA  
2 VALUES ('PR02', 'Elena', 'Rojas', 'Urdesa', 'Guayas', '042333444');
```

1 fila creada.

```
SQL> INSERT INTO PROV_TIENDA  
2 VALUES ('PR03', 'Luis', 'Castro', 'Letamendi', 'Tungurahua', '032555666');
```

1 fila creada.

```
SQL> INSERT INTO COMPRA_TIENDA  
2 VALUES (1001, 'P10', 'C001', TO_DATE('2026-03-28', 'YYYY-MM-DD'),  
'Tarjeta');
```

1 fila creada.

```
SQL> INSERT INTO COMPRA_TIENDA  
2 VALUES (1001, 'P11', 'C001', TO_DATE('2026-03-28', 'YYYY-MM-DD'),  
'Tarjeta');
```

1 fila creada.

```
SQL> INSERT INTO COMPRA_TIENDA
```



2 VALUES (1002, 'P12', 'C002', TO_DATE('2026-03-29', 'YYYY-MM-DD'), 'Efectivo');

1 fila creada.

SQL> INSERT INTO ENTREGA_TIENDA

2 VALUES (501, 'P10', 'PR01', TO_DATE('2026-03-15', 'YYYY-MM-DD'), 10, 110.00);

1 fila creada.

SQL> INSERT INTO ENTREGA_TIENDA

2 VALUES (502, 'P11', 'PR02', TO_DATE('2026-03-18', 'YYYY-MM-DD'), 20, 25.00);

1 fila creada.

SQL> INSERT INTO ENTREGA_TIENDA

2 VALUES (503, 'P12', 'PR03', TO_DATE('2026-03-20', 'YYYY-MM-DD'), 30, 12.00);

1 fila creada.

SQL> COMMIT;

EJERCICIO 4

Enunciado

"Una empresa necesita organizar la siguiente información referente a su organización interna.

La empresa está organizada en una serie de departamentos. Cada departamento tiene un código, nombre y presupuesto anual. Cada departamento está ubicado en un centro de trabajo y el centro de trabajo se compone de varios departamentos. La información que se desea guardar del centro de trabajo es el código de centro, nombre, población y dirección del centro.

La empresa tiene una serie de empleados. Cada empleado tiene un teléfono, fecha de alta en la empresa, c1i y nombre y apellido y el salario.

A esta empresa también le interesa tener guardada información sobre los hijos de los empleados. Cada hijo de un empleado tendrá un código, nombre, apellido y fecha de nacimiento.

Se desea mantener también información sobre las habilidades de los empleados (¿por ejemplo, mercadotecnia, trato con el cliente, fresador, operador de telefonía, etc). Cada habilidad tendrá una descripción y un código.

Sobre este supuesto diseñar la bd, teniendo en cuenta los siguientes aspectos.



Un departamento estará compuesto por uno o más empleados. Un empleado está asignado a un único departamento.

El centro de trabajo se compone de varios departamentos, cada departamento se ubica en un único centro de trabajo.

Un empleado puede tener varios hijos. 1 hijo pertenece a 1 solo empleado.

Un empleado puede tener varias habilidades, y una misma habilidad puede ser poseída por empleados diferentes en diferente porcentaje.

1. Análisis del problema

Se identifican las entidades principales:

- **Centro**
- **Departamento**
- **Empleado**
- **Hijo**
- **Habilidad**

Entidad intermedia:

- **Emp_Hab**

Porque:

- Empleado ↔ Habilidad → relación **N:M**
- Se necesita una tabla intermedia con atributo adicional (porcentaje)

2. Modelo conceptual

2.1 Entidades del negocio

Centro

Atributos:

- Cod_Centro
- Nombre
- Población
- Dirección

Departamento

Atributos:

- Cod_Dep
- Nombre
- Presupuesto



- Cod_Centro

Empleado

Atributos:

- CI
- Nombre
- Apellido
- Teléfono
- Fecha_Alta
- Salario
- Cod_Dep

Hijo

Atributos:

- Cod_Hijo
- Nombre
- Apellido
- Fecha_Nac
- CI_Empleado

Habilidad

Atributos:

- Cod_Hab
- Descripción

Emp_Hab

Atributos:

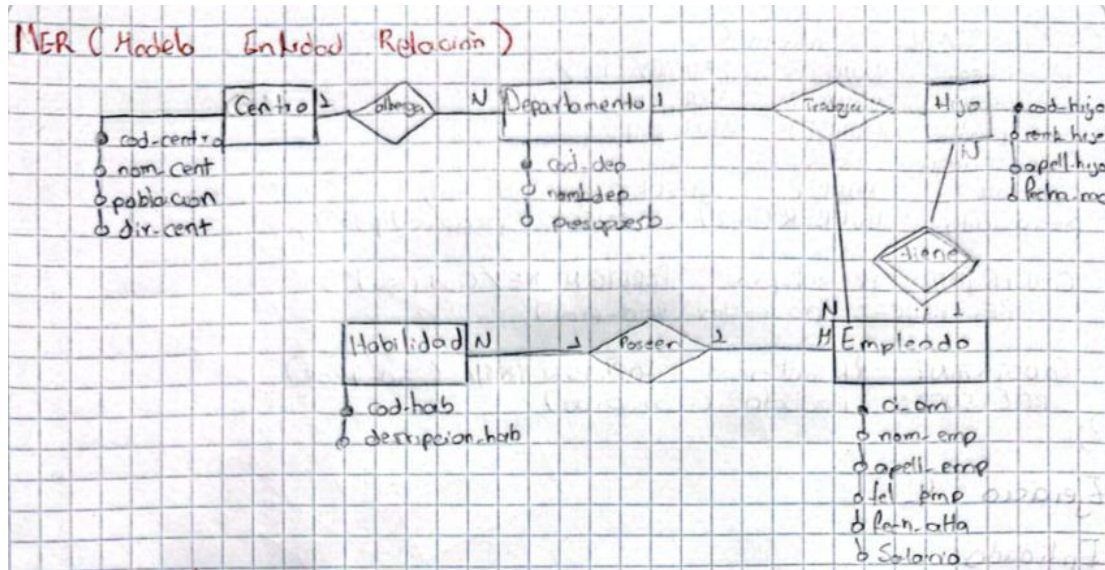
- CI_Empleado
- Cod_Hab
- Porcentaje

2.2 Reglas de negocio

1. Un centro tiene varios departamentos
2. Un departamento pertenece a un solo centro
3. Un departamento tiene varios empleados
4. Un empleado pertenece a un solo departamento
5. Un empleado puede tener varios hijos
6. Un hijo pertenece a un solo empleado
7. Un empleado puede tener varias habilidades



8. Una habilidad puede pertenecer a varios empleados



3. Modelo Entidad-Relación (ER)

3.1 Entidades con claves

CENTRO

- Cod_Centro (PK)
- Nombre
- Población
- Dirección

DEPARTAMENTO

- Cod_Dep (PK)
- Nombre
- Presupuesto
- Cod_Centro (FK)

EMPLEADO

- CI (PK)
- Nombre
- Apellido
- Teléfono
- Fecha_Alta
- Salario
- Cod_Dep (FK)

HIJO



- Cod_Hijo (PK)
- Nombre
- Apellido
- Fecha_Nac
- CI_Empleado (FK)

HABILIDAD

- Cod_Hab (PK)
- Descripción

EMP_HAB

- CI_Empleado (PK, FK)
- Cod_Hab (PK, FK)
- Porcentaje

3.2 Relaciones y cardinalidades

CENTRO — TIENE — DEPARTAMENTO

- Un centro tiene varios departamentos
- Un departamento pertenece a un centro

CENTRO (1) ----- (N) DEPARTAMENTO

DEPARTAMENTO — TIENE — EMPLEADO

- Un departamento tiene varios empleados
- Un empleado pertenece a un departamento

DEPARTAMENTO (1) ----- (N) EMPLEADO

EMPLEADO — TIENE — HIJO

- Un empleado puede tener varios hijos
- Un hijo pertenece a un solo empleado

EMPLEADO (1) ----- (N) HIJO

EMPLEADO — POSEE — HABILIDAD

- Un empleado tiene varias habilidades
- Una habilidad pertenece a varios empleados

EMPLEADO (N) ----- (M) HABILIDAD

(resuelto con EMP_HAB)

Paso del modelo ER al modelo relacional



Regla 1: Entidades → Tablas

- CENTRO
- DEPARTAMENTO
- EMPLEADO
- HIJO
- HABILIDAD
- EMP_HAB

Regla 2: Relación 1:N

- CENTRO → DEPARTAMENTO
FK: cod_centro
- DEPARTAMENTO → EMPLEADO
FK: cod_dep
- EMPLEADO → HIJO
K: ci_empleado

Regla 3: Relación N:M

EMPLEADO ↔ HABILIDAD

Se crea:

EMP_HAB

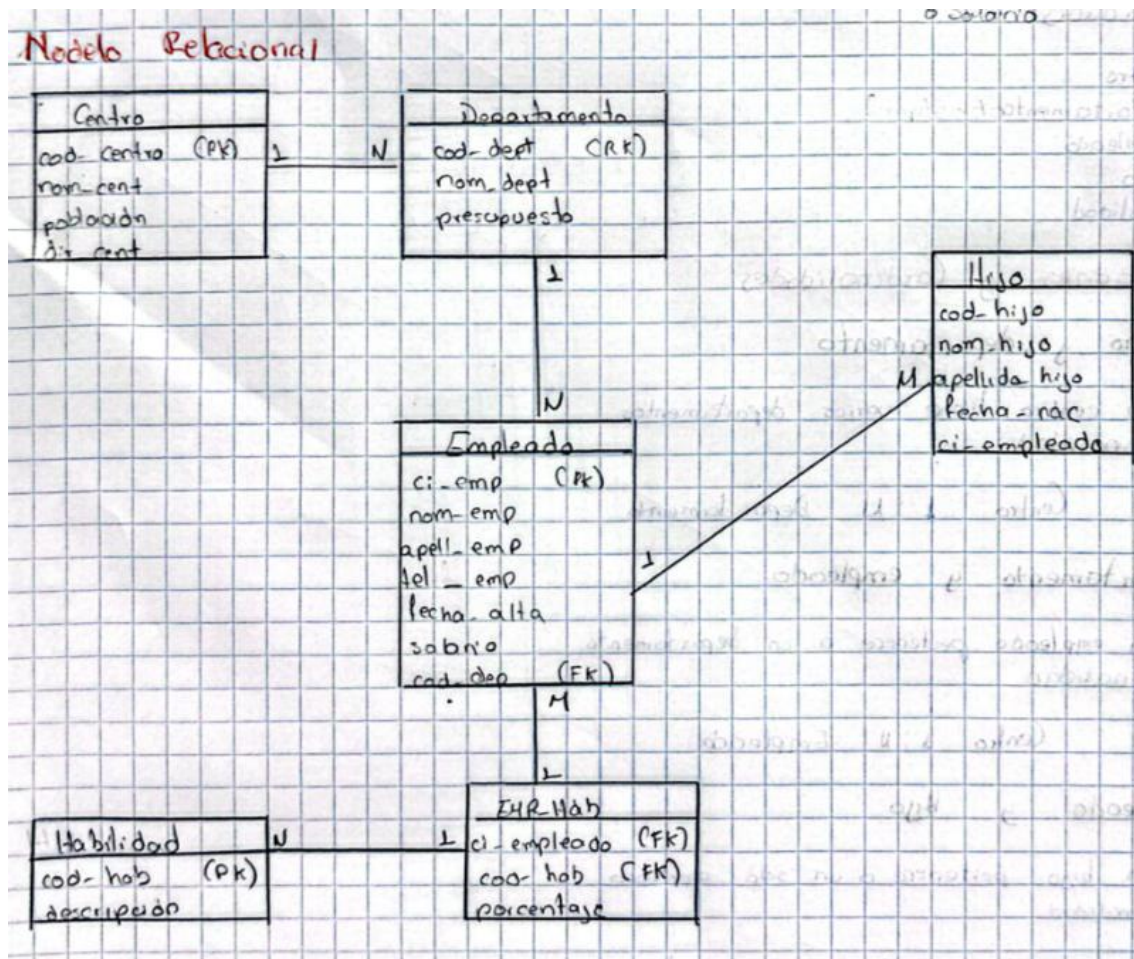
Con:

- CI_EMPLEADO
- COD_HAB
- PORCENTAJE

PK compuesta:

(CI_EMPLEADO, COD_HAB)

5. Modelo relacional



6. Modelo físico en Oracle (DDL)

```
CREATE TABLE CENTRO (  
    cod_centro NUMBER PRIMARY KEY,  
    nombre VARCHAR2(100) NOT NULL,  
    poblacion VARCHAR2(50),  
    direccion VARCHAR2(100)  
);
```

```
CREATE TABLE DEPARTAMENTO (  
    cod_dep NUMBER PRIMARY KEY,  
    nombre VARCHAR2(100) NOT NULL,  
    presupuesto NUMBER(12,2) CHECK (presupuesto > 0),  
    cod_centro NUMBER NOT NULL,  
    CONSTRAINT fk_dep_cen FOREIGN KEY (cod_centro)  
        REFERENCES CENTRO(cod_centro)  
);
```

```
CREATE TABLE EMPLEADO (  
    ci VARCHAR2(10) PRIMARY KEY,  
    nombre VARCHAR2(50) NOT NULL,  
    apellido VARCHAR2(50) NOT NULL,  
    telefono VARCHAR2(15),
```



```
    fecha_alta DATE,  
    salario NUMBER(10,2) CHECK (salario > 0),  
    cod_dep NUMBER NOT NULL,  
    CONSTRAINT fk_emp_dep FOREIGN KEY (cod_dep)  
        REFERENCES DEPARTAMENTO(cod_dep)  
);  
  
CREATE TABLE HIJO (  
    cod_hijo NUMBER PRIMARY KEY,  
    nombre VARCHAR2(50) NOT NULL,  
    apellido VARCHAR2(50),  
    fecha_nac DATE,  
    ci_empleado VARCHAR2(10) NOT NULL,  
    CONSTRAINT fk_hijo_emp FOREIGN KEY (ci_empleado)  
        REFERENCES EMPLEADO(ci)  
);  
  
CREATE TABLE HABILIDAD (  
    cod_hab NUMBER PRIMARY KEY,  
    descripcion VARCHAR2(200) NOT NULL  
);  
  
CREATE TABLE EMP_HAB (  
    ci_empleado VARCHAR2(10) NOT NULL,  
    cod_hab NUMBER NOT NULL,  
    porcentaje NUMBER(5,2) CHECK (porcentaje BETWEEN 0 AND  
100),  
    CONSTRAINT pk_emp_hab PRIMARY KEY (ci_empleado, cod_hab),  
    CONSTRAINT fk_ah_emp FOREIGN KEY (ci_empleado)  
        REFERENCES EMPLEADO(ci),  
    CONSTRAINT fk_ah_hab FOREIGN KEY (cod_hab)  
        REFERENCES HABILIDAD(cod_hab)  
);
```

8. DML (Inserciones)

CENTRO

```
INSERT INTO CENTRO VALUES (1,'Centro Norte','Quito','Av.  
Amazonas');
```

DEPARTAMENTO

```
INSERT INTO DEPARTAMENTO VALUES (1,'Sistemas',50000,1);
```

EMPLEADO

```
INSERT INTO EMPLEADO VALUES  
('0101','Juan','Perez','099999999',SYSDATE,1200,1);
```

HIJO



```
INSERT INTO HIJO VALUES (1, 'Pedro', 'Perez', DATE '2015-06-10', '0101');
```

HABILIDAD

```
INSERT INTO HABILIDAD VALUES (1, 'Programacion');
```

EMP_HAB

```
INSERT INTO EMP_HAB VALUES ('0101', 1, 85);
```

Confirmar

```
COMMIT;
```

9. Consulta integrada

```
SELECT
    e.nombre,
    e.apellido,
    d.nombre AS departamento,
    h.descripcion,
    eh.porcentaje
FROM EMPLEADO e
JOIN DEPARTAMENTO d ON e.cod_dep = d.cod_dep
JOIN EMP_HAB eh ON e.ci = eh.ci_empleado
JOIN HABILIDAD h ON eh.cod_hab = h.cod_hab;
```

10. Operaciones DML

UPDATE

```
UPDATE EMPLEADO SET salario = 1500 WHERE ci='0101';
UPDATE DEPARTAMENTO SET presupuesto = 60000 WHERE
cod_dep=1;
```

DELETE

```
DELETE FROM HIJO WHERE cod_hijo = 1;
```

11.SCRYPT

```
SQL> CREATE USER ORGANIZACION_INTERNA IDENTIFIED BY
ORGANIZACION123;
```

Usuario creado.

```
SQL> GRANT CONNECT, RESOURCE, UNLIMITED TABLESPACE TO
ORGANIZACION_INTERNA;
```

Concesión terminada correctamente.



SQL> CONNECT ORGANIZACION_INTERNA;

Introduzca la contraseña:

Conectado.

```
SQL> CREATE TABLE CENTRO_TRABAJO
2 (COD_CENTRO VARCHAR2(10) PRIMARY KEY,
3  NOM_CENTRO VARCHAR2(50) NOT NULL,
4  POB_CENTRO VARCHAR2(50),
5  DIR_CENTRO VARCHAR2(100));
```

Tabla creada.

```
SQL> CREATE TABLE DEPARTAMENTO
2 (COD_DEP VARCHAR2(10) PRIMARY KEY,
3  NOM_DEP VARCHAR2(50) NOT NULL,
4  PRESUP_DEP NUMBER(12,2) CHECK (PRESUP_DEP > 0),
5  COD_CENTRO VARCHAR2(10),
6  CONSTRAINT FK_DEP_CENTRO FOREIGN KEY (COD_CENTRO) REFERENCES
CENTRO_TRABAJO(COD_CENTRO));
```

Tabla creada.

```
SQL> CREATE TABLE EMPLEADO
2 (CI_EMP VARCHAR2(10) PRIMARY KEY,
3  NOM_EMP VARCHAR2(50) NOT NULL,
4  APE_EMP VARCHAR2(50) NOT NULL,
5  TEL_EMP VARCHAR2(15),
6  FEC_ALTA_EMP DATE,
7  SALARIO_EMP NUMBER(10,2) CHECK (SALARIO_EMP > 0),
8  COD_DEP VARCHAR2(10),
9  CONSTRAINT FK_EMP_DEP FOREIGN KEY (COD_DEP) REFERENCES
DEPARTAMENTO(COD_DEP));
```

Tabla creada.

```
SQL> CREATE TABLE HIJO
2 (COD_HIJO VARCHAR2(10) PRIMARY KEY,
3  NOM_HIJO VARCHAR2(50) NOT NULL,
4  APE_HIJO VARCHAR2(50) NOT NULL,
5  FEC_NAC_HIJO DATE,
6  CI_EMP VARCHAR2(10),
7  CONSTRAINT FK_HIJO_EMP FOREIGN KEY (CI_EMP) REFERENCES
EMPLEADO(CI_EMP));
```

Tabla creada.

```
SQL> CREATE TABLE HABILIDAD
2 (COD_HAB VARCHAR2(10) PRIMARY KEY,
3  DESC_HAB VARCHAR2(100) NOT NULL);
```

Tabla creada.

```
SQL> CREATE TABLE EMPLEADO_HABILIDAD
2 (CI_EMP VARCHAR2(10),
3  COD_HAB VARCHAR2(10),
4  PORCENTAJE NUMBER(3) CHECK (PORCENTAJE >= 0 AND PORCENTAJE <= 100),
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO– JULIO 2026



5 PRIMARY KEY (CI_EMP, COD_HAB),
6 CONSTRAINT FK_EH_EMP FOREIGN KEY (CI_EMP) REFERENCES
EMPLEADO(CI_EMP),
7 CONSTRAINT FK_EH_HAB FOREIGN KEY (COD_HAB) REFERENCES
HABILIDAD(COD_HAB));

Tabla creada.

SQL> COMMIT;

Confirmación terminada.

SQL> INSERT INTO CENTRO_TRABAJO
2 VALUES ('CT01', 'Matriz Norte', 'Quito', 'Av. Naciones Unidas');

1 fila creada.

SQL> INSERT INTO CENTRO_TRABAJO
2 VALUES ('CT02', 'Sucursal Sur', 'Guayaquil', '9 de Octubre');

1 fila creada.

SQL> INSERT INTO CENTRO_TRABAJO
2 VALUES ('CT03', 'Planta Central', 'Ambato', 'Parque Industrial');

1 fila creada.

SQL> INSERT INTO DEPARTAMENTO
2 VALUES ('D10', 'Gerencia', 150000, 'CT01');

1 fila creada.

SQL> INSERT INTO DEPARTAMENTO
2 VALUES ('D20', 'Ventas', 80000, 'CT02');

1 fila creada.

SQL> INSERT INTO DEPARTAMENTO
2 VALUES ('D30', 'Produccion', 120000, 'CT03');

1 fila creada.

SQL> INSERT INTO EMPLEADO
2 VALUES ('1711', 'Juan', 'Perez', '09911', TO_DATE('2018-02-15', 'YYYY-MM-DD'), 1200,
'D10');

1 fila creada.

SQL> INSERT INTO EMPLEADO
2 VALUES ('0922', 'Maria', 'Lopez', '09822', TO_DATE('2020-06-10', 'YYYY-MM-DD'), 900,
'D20');

1 fila creada.

SQL> INSERT INTO EMPLEADO



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO– JULIO 2026



2 VALUES ('1833', 'Carlos', 'Ruiz', '09733', TO_DATE('2019-11-20', 'YYYY-MM-DD'), 1100, 'D30');

1 fila creada.

SQL> INSERT INTO HIJO

2 VALUES ('H1', 'Lucas', 'Perez', TO_DATE('2015-05-12', 'YYYY-MM-DD'), '1711');

1 fila creada.

SQL> INSERT INTO HIJO

2 VALUES ('H2', 'Sofia', 'Perez', TO_DATE('2019-08-25', 'YYYY-MM-DD'), '1711');

1 fila creada.

SQL> INSERT INTO HIJO

2 VALUES ('H3', 'Mateo', 'Ruiz', TO_DATE('2021-12-05', 'YYYY-MM-DD'), '1833');

1 fila creada.

SQL> INSERT INTO HABILIDAD

2 VALUES ('HB1', 'Trato con el cliente');

1 fila creada.

SQL> INSERT INTO HABILIDAD

2 VALUES ('HB2', 'Operador de maquinaria pesada');

1 fila creada.

SQL> INSERT INTO HABILIDAD

2 VALUES ('HB3', 'Mercadotecnia digital');

1 fila creada.

SQL> INSERT INTO EMPLEADO_HABILIDAD

2 VALUES ('1711', 'HB1', 90);

1 fila creada.

SQL> INSERT INTO EMPLEADO_HABILIDAD

2 VALUES ('0922', 'HB1', 100);

1 fila creada.

SQL> INSERT INTO EMPLEADO_HABILIDAD

2 VALUES ('1833', 'HB2', 85);

1 fila creada.

SQL> COMMIT;